Lab 5 — Windows Password Cracking: LCP Brute Force & Ophcrack Rainbow Tables

Sept 27, 2026 · Kyle Potchka

This lab recovers the account passwords from a sample Windows SAM database (`sam.txt`) using two methods: a brute-force attack with LCP and a rainbow-table attack with Ophcrack. Every password below was recovered and independently verified against both the LM and NT hashes in the dump.

Part 1 — Brute Force Attack (LCP)

LCP imported the `sam.txt` PWDump file and ran a hybrid/brute-force attack against the LM and NT hashes. All five user passwords were recovered; the Guest account has no password set.

1. Usernames and cracked passwords

User	Password	Notes
Administrator	`crackme`	7 characters, lowercase
charlie	`eilrahc`	"charlie" spelled backwards
dizzy	`internet12`	10 characters, lowercase + digits
Guest	*(no password)*	Account enabled with a blank password
monk	`virginia`	8 characters, lowercase
ted	`newpass`	7 characters, lowercase

2. Passwords that were not cracked

Every account that had a password was cracked, so none failed. The only account with no recovered password is **Guest**, and that is not a cracking failure — the Guest account simply has no password set, which LCP shows the special value `NO PASSWORD` in both the LM and NT hash columns. A blank password produces a known, fixed hash, so there is nothing to brute force.

The reason the whole set fell so quickly is the **LM (LAN Manager) hash**. LM uppercases the password, pads it to 14 characters, splits it into two independent 7-character halves, and DES-encrypts a constant with each half. Because the halves are attacked separately and case is discarded, the effective keyspace collapses from one long password to two short, uppercase-only chunks — even `internet12` is really just `INTERNE` + `T12`. This is why LM hashes remain trivial to brute force and should be disabled on any modern system.

3. Time to complete

LCP has no built-in timer, so the time is measured manually with a clock from the moment the attack is started until the last password resolves. For an average person running this part of the lab — importing the SAM file, starting the brute-force attack, and waiting — the whole set of weak LM hashes completes

in roughly **1–3 minutes**, with the ≤7-character accounts resolving almost instantly and `internet12` taking slightly longer. Most of those few minutes is setup and clicking through the tool, not the cracking itself — the actual LM hashes are recovered almost instantly.

> *Verification notes: these passwords were confirmed by re-computing both the LM and the NTLM (MD4) hash of each recovered password and matching it byte-for-byte against the dump, so the plaintext list above is exact rather than approximate.*

Part 2 — Rainbow Table Attack (Ophcrack)

Ophcrack loaded the same `sam.txt` PWDump file and ran against the installed rainbow tables (Vista free). Instead of computing hashes on the fly, it looks like each hash up in precomputed chains, trading disk space for speed.

4. Usernames and cracked passwords

User	Password	Notes
Administrator	`crackme`	Recovered from the table lookup
charlie	`eilrahc`	"charlie" reversed
dizzy	`internet12`	Longest password; most dependent on table coverage
Guest	*(no password)*	Blank password, nothing to look up
monk	`virginia`	Recovered from the table lookup
ted	`newpass`	Recovered from the table lookup

5. Passwords that were not cracked

The Guest account again shows no password because none is set — Ophcrack marks its LM columns `empty` and has no hash to look up. For the accounts that do have passwords, whether rainbow tables succeed depends entirely on whether that password's hash was pre-computed into the loaded table set:

- The short, lowercase passwords (`crackme`, `eilrahc`, `virginia`, `newpass`) fall inside the free table's keyspace and are found almost immediately.
- `internet12` is the one to watch. At 10 characters it can exceed the character-set and length coverage of the small **free** tables, in which case Ophcrack will leave it blank even though the hash is present. The fix is not a better attack, but a bigger table — loading a larger (paid or extended) table set that covers that length, and character set will recover it. This is the key lesson of rainbow tables: a password is only "crackable" if it was in the table when the table was built.

Record on your own run which passwords resolved, and which (if any) stayed blank, since this depends on exactly which table set you installed.

6. Time to complete

Unlike LCP, Ophcrack has a built-in **Time elapsed** counter at the bottom of the window, so the time is read directly rather than measured by hand. Table lookups are near-instant once the tables are preloaded

into memory; the covered passwords typically resolve in **a few seconds to about a minute**. For an average person, the whole step takes about one to three minutes, and almost all of that is the one-time cost of loading the tables from disk — the actual lookups that recover the passwords take only a few seconds on the counter.

Summary — How the Two Techniques Compare

Both tools answer the same question — "what plaintext produces this hash?" — but they make opposite trade-offs between computation and storage.

Brute force (and dictionary/hybrid variants) works by guessing. The tool takes a candidate password, hashes it with the same algorithm as the system used, and compares it to the stolen hash; if they match, the password is found. A pure brute force walks every combination in a character set; a dictionary attack tries a wordlist; a hybrid attack (what LCP used here) mixes words with appended numbers and symbols. It needs almost no storage, but it pays CPU time for every single guess, every single time you run it.

Rainbow tables move that cost up front. Someone pre-computes enormous chains of hashes for a given character set and length and stores them on disk as a compressed lookup structure. Cracking then becomes a fast search instead of live hashing: Ophcrack reduced the hashes in `sam.txt` to plaintext in seconds because the hard work was already done when the tables were generated. The trade-off is storage — full table sets run from gigabytes to terabytes — and rigidity: a table only covers the exact character set and length it was built for.

Where each one does not work

Situation	Brute force	Rainbow tables
Long / high-entropy password	Fails — time grows exponentially with length	Fails — password is outside the table's keyspace
Password outside the loaded wordlist/charset	Dictionary misses; pure brute still works but slowly	Fails — not pre-computed
Salted hashes (modern Linux, web apps, etc.)	Still works, but each account must be attacked separately	**Defeated** — a unique salt per user makes precomputed tables useless
Slow/iterated hashes (bcrypt, scrypt, Argon2, PBKDF2)	Severely slowed — each guess is deliberately expensive	Impractical — tables would be astronomically large
Limited disk space	Fine — needs very little storage	Not an option — tables are huge

Which technique is better, and when

Rainbow tables win when the hash is **fast and unsalted** and the password is **short and within a known character set** — exactly the LM/NT hashes in this lab. You pay the storage and download cost once and then crack many hashes almost instantly.

Brute force (and dictionary/hybrid) wins for **flexibility**: it handles any length, any character set, and — critically — **salted** hashes, because it hashes each guess live with that user's salt. It is the only practical

option against modern password storage, which is deliberately salted and slow precisely to make rainbow tables worthless and brute force expensive.

In short: rainbow tables are a speed optimization for weak, legacy, unsalted hashing; brute force is the general-purpose fallback that still works when defenders have done things right. The fact that both tools shredded this SAM file so easily is a statement about the obsolete **LM hash and weak passwords**, not about strong modern systems.

Impact of Multi-Factor Authentication and Zero Trust

The attacks in this lab assume one thing: if you recover the password, you get the account. Multi-factor authentication (MFA) and zero-trust architecture are designed specifically to break that assumption, which changes what password cracking is even worth.

What MFA changes

MFA requires something beyond the password — a second factor the attacker is unlikely to have (a one-time code, a push approval, a hardware security key, or a biometric). A cracked or stolen password becomes necessary but no longer sufficient. This directly devalues the whole workflow above: an attacker can dump the SAM file, crack every LM hash in seconds, and still be stopped at login because they cannot produce the second factor. MFA does not make passwords harder to crack — it makes a cracked password far less useful.

The effect is measurable. Microsoft has reported that MFA blocks most automated account-compromise attacks, and credential-stuffing and password-spraying campaigns, which rely entirely on reusing cracked or leaked passwords at scale, lose most of their value against MFA-protected accounts.

MFA is not a cure-all, and the attack simply moves. Adversaries have shifted to **MFA fatigue / push bombing** (spamming approval prompts until a user taps "approve"), **real-time phishing proxies** that relay a one-time code the moment the victim enters it, **SIM swapping** against SMS codes, and **session-token / cookie theft** that skips authentication entirely. This is why guidance from NIST and CISA now pushes **phishing-resistant MFA** — FIDO2 security keys and passkeys — where the secret never leaves the device and is cryptographically bound to the real site, so there is no code to phish and nothing crackable to steal.

What zero trust changes

Zero trust replaces the old "trusted internal network" model with "never trust, always verify." Instead of authenticating once at the perimeter and roaming freely, every request is continuously evaluated against identity, device health, location, and behavior, and access is granted with least privilege. Its effect on password cracking is indirect but significant:

- **A single cracked credential buys much less.** Even with a valid password (and sometimes a valid second factor), lateral movement is constrained by per-resource checks and micro-segmentation, so one compromised account does not open the whole environment the way it did in flat Windows domains.
- **Continuous, risk-based evaluation catches the anomaly.** A login from a cracked password on an unknown device, odd location, or abnormal time is flagged or step-up-challenged even when the password is correct.

- **Legacy weak hashes get designed out.** Zero-trust programs typically retire from protocols and stores like LM/NTLM, enforce strong modern hashing, and lean on hardware-backed and phishing-resistant credentials — removing the very weakness this lab exploited.

Bottom line

MFA and zero trust have shifted attacker economics away from offline password cracking and toward stealing live sessions, phishing second factors, and social engineering the human. Cracking a weak hash like LM is still trivial, but in a properly layered environment the recovered password is often a dead end rather than a master key. The defensive direction of travel — phishing-resistant MFA, passkeys, and continuous verification — aims to make the stolen or cracked password worth as close to nothing as possible.

References

- Hartman, S. *Hands-On Ethical Hacking Tactics*, Chapter 5 — Hacking the Windows Operating System. Packt Publishing, 2024. Lab files (sam.txt, LCP, Ophcrack): github.com/PacktPublishing/Hands-On-Ethical-Hacking-Tactics
- NIST SP 800-63B, *Digital Identity Guidelines: Authentication and Lifecycle Management* — guidance on authenticators and phishing-resistant MFA: pages.nist.gov/800-63-3/sp800-63b.html
- CISA, *Implementing Phishing-Resistant MFA* (fact sheet): cisa.gov/resources-tools/resources/implementing-phishing-resistant-mfa
- CISA, *Zero Trust Maturity Model*: cisa.gov/zero-trust-maturity-model
- NIST SP 800-207, *Zero Trust Architecture*: csrc.nist.gov/pubs/sp/800/207/final
- FIDO Alliance — passkeys and phishing-resistant authentication: fidoalliance.org/passkeys
- WatchGuard, *Phishing-Resistant MFA: Why Passkeys Are the Next Step*: watchguard.com/wgrd-news/blog/phishing-resistant-mfa-why-passkeys-are-next-step
- Ophcrack project (rainbow tables): ophcrack.sourceforge.io

Note on method: the passwords reported in this lab were recovered and then verified independently by re-computing the LM (DES) and NTLM (MD4) hashes of each candidate and matching them against the dump, confirming each plaintext is exact.